Introduction to machine learning

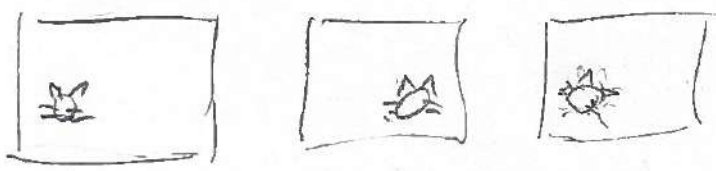
- 8 -

Problems with FFNNs and images:

too many weights

e.g. 1MPixel RGB (640 × 480) input 300 neurons output, no hidden layer

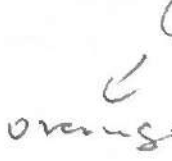
- 10^9 parameters. Simply too many
  - training size
  - training time
  - etc
- positional dependence.



all different.

When I draw an image, I do not need colour, just the contours (mostly)





Actually our biological eyes do this

$$f(t) * g(t) = \int_{-\infty}^{\infty} f(\tau)\, g(t-\tau)\, d\tau$$

or if $t \in [t_1, t_2]$: $f(t)$

$$f(t) * s(t) = \int_{t_1}^{t_2} f(\tau)\, s(t-\tau)\, d\tau$$

Discrete:

Discrete:

$$(f * g)(n) = \sum_{m=-\infty}^{\infty} f(m)\, g(n-m)$$

again if f is only defined for $n = 0, \dots N-1$

then

$$(f * g)(n) = \sum_{m=0}^{N-1} f(m)\, g(n-m), \text{ or } \left| (f*g)(n) = \sum_{m=-N}^{N} f(m) g(n-m) \right. \quad \left. n = -N, \dots -1, 0, 1, \dots N \right|$$

Examples: 1) $f = [0, 1, 0]$ for $n = -1, 0, 1$

$$(f * g)(n) = \sum_{m=-1}^{1} f(m)\, g(n-m) = f(0)\, g(n-0) = g(n)$$

Identity: $f = [0 \; 0 \; 0 \dots 0, 1, 0 \dots 0 \; 0 \; 0]$

2) $f = [-1/2, 0, 1/2]$ for $n = -1, 0, 1$

$$(f * g)(n) = -\frac{1}{2} g(n-1) + \frac{1}{2} g(n+1) = \frac{g(n+1) - g(n-1)}{2}$$

derivative!! Sensitive only for changes in g.

3) Very often we do not care about the size of change (black or white cat), or cont. grad.

$$f = [-1, 2, -1]$$

$$(f * g)(n) = g(n+1) - 2g(n) + g(n-1)$$

this is the second derivative. This is only sensitive to sudden changes in g

Two dimensional f is called Kernel

$$(f * g)(n,m) = \sum_{i=-N}^{N} \sum_{j=-M}^{M} f(i,j)\, g(n-i, m-j)$$

Examples:

1.) Identity $f = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$

2.) 2d Laplace: $f = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}$

$$(f * g)(n,m) = 4g(n,m) - g(n-1,m) - g(n+1,m) - g(n,m-1) - g(n,m+1)$$

called: edge detection

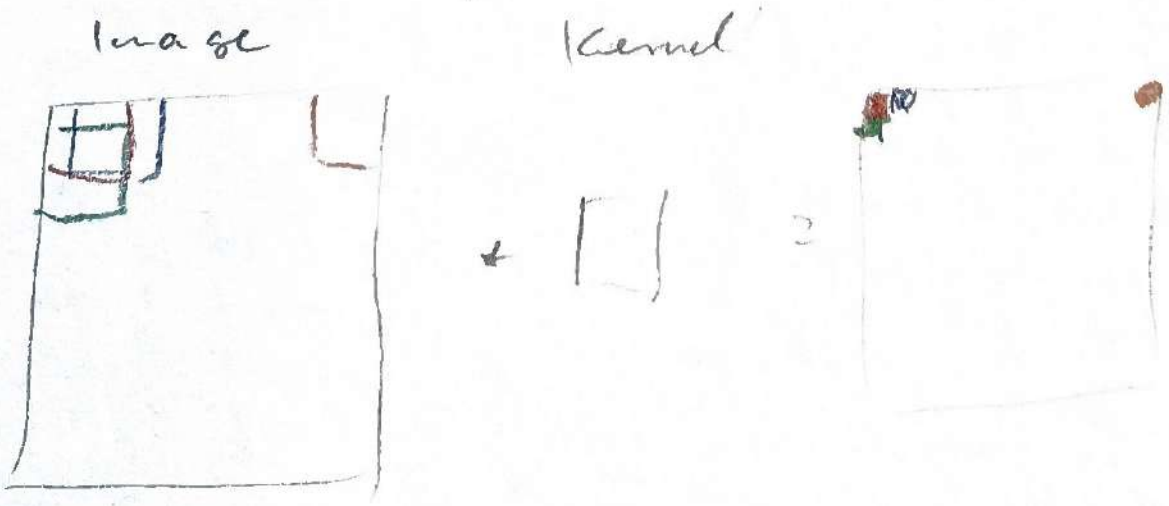
3.) enhanced edge detection

$$f = \begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}$$

4.) blur: box $\frac{1}{9}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$ Gaussian $\frac{1}{16}\begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$

5.) Important can be of any size!

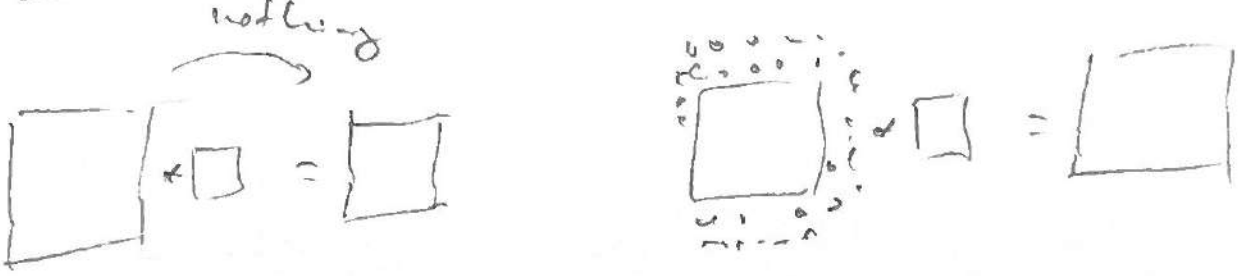
Convolution of an image:



Padding: What to do with the edges

- Nothing: image size shrinks by 2, 4, 6...
  (can be problematic with pooling see below)

add:
- Zero: large signal at boundary
- same: the same value as the last pixel



Striding: - Colors change only gradually
- reduce image size before FINAL

Move the convolution matrix by 'stride' steps.

3D Activation: images have colours e.g. RGB

One kernel matrix for each layer
(plus one bias for each)

$$Z^{(l)}_{n,m,c} = \sum_{i,j} K^{(l)}_{i,j,c} A^{(l-1)}_{n-i,m-j,c} + b^{(l)}_c$$

How to determine $K_{i,j}$?
We will not try to be smart but let the computer do it:

$$\frac{\partial \mathcal{L}}{\partial K^{(l)}_{i,j}} = \sum_{c,d} \frac{\partial \mathcal{L}}{\partial Z^{(l)}_{c,d}} \frac{\partial Z^{(l)}_{c,d}}{\partial K^{(l)}_{i,j}}$$

c, d runs for the convoluted image

$$\frac{\partial \mathcal{L}}{\partial K^{(l)}_{i,j}} = \sum_{c,d} \frac{\partial \mathcal{L}}{\partial Z^{(l)}_{c,d}} A^{(l-1)}_{n-i,m-j} = A^{(l-1)} \circledast \frac{\partial \mathcal{L}}{\partial Z^{(l)}}$$

Max pooling

- downsize the image
- position is not that important
- high convolution results are important

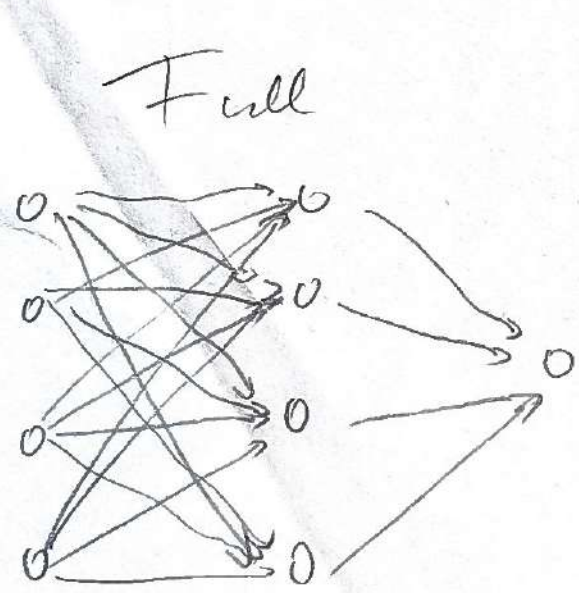
1	3	5	4
9	8	3	2
0	0	0	1
1	1	1	2

Max pooling →

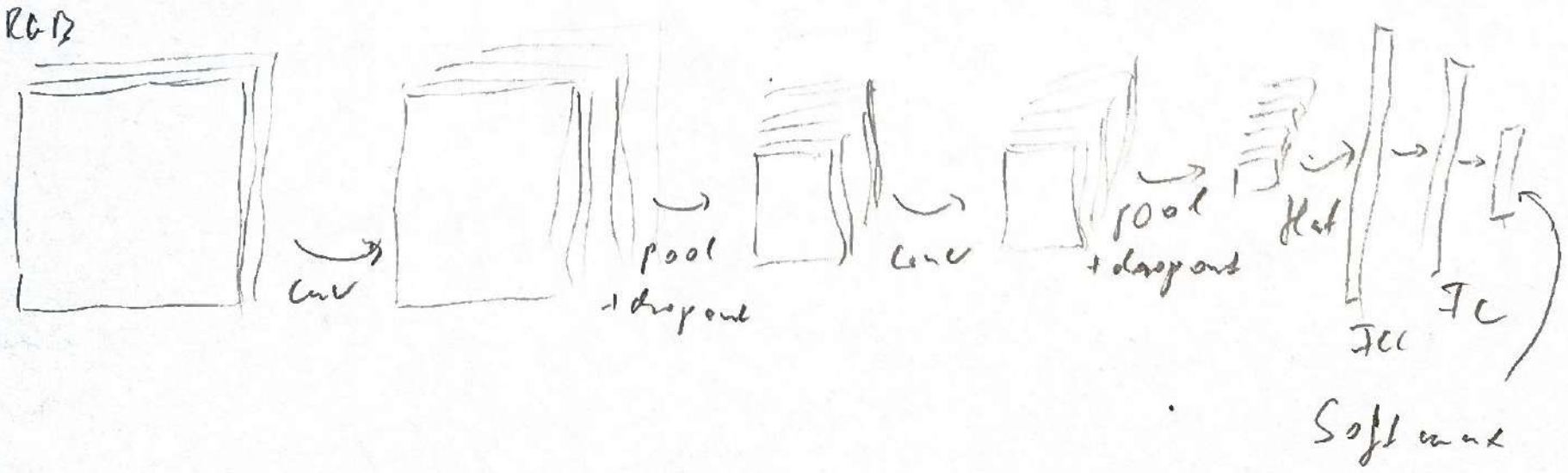
9	5
1	2

Dropout

- regularization by temporarily removing the network



Convolutional neural network typical architecture



Optimizers

Gradient - Minibatch - Stochastic Gradient

→ noise
- escape local minima
- unnecessary oscillations

Momentum

- move in the direction of the actual gradient and the average of the previous ones

$$\Delta w(t) = \alpha \cdot \Delta w(t-1) + \eta \frac{\partial \mathcal{L}}{\partial w}(t)$$

α: friction or dampening parameter; η: learning rate

Implementation:

learning rate ↓

$$\Delta w = \underbrace{\text{momentum}}_{\alpha} \cdot \Delta w + (1 - \underbrace{\text{dampening}}_{\alpha}) \cdot \eta \cdot \text{grad}\, w$$

$$w = w - \Delta w$$

→ 2 parameters

Adaptive learning rates

$$\Delta w_{ij} = \eta \, g_{ij} \frac{\partial \mathcal{L}}{\partial w_{ij}} \qquad g(t=0) = 0$$

g_{ij} a weight based learning rate

If gradient is consistent: $g_{ij}(t) = g_{ij}(t-1) + \beta$

otherwise: $g_{ij}(t) = g_{ij}(t-1) \cdot (1-\beta)$

Ex.: RMS Prop

Moving average of the squared gradients:

$$E[s^2](t) = \beta\, E[s^2](t-1) + (1-\beta)\left(\frac{\partial \alpha}{\partial w}\right)^2$$

$$w_{ij}(t) = w_{ij}(t-1) - \frac{\gamma}{\sqrt{E[s](t)}} \frac{\partial \alpha}{\partial w_{ij}}$$

ADAM: combination of Momentum and RMS Prop

$$m_w(t+1) = \beta_1 m_w(t) + (1-\beta_1)\frac{\partial \alpha}{\partial w}(t)$$

$$v_w(t+1) = \beta_2 v_w(t) + (1-\beta_2)\left(\frac{\partial \alpha}{\partial w}(t)\right)^2$$

$$\hat{m}_w = \frac{m_w(t+1)}{1-\beta_1}$$

$$\hat{v}_w = \frac{v_w(t+1)}{1-\beta_2}$$

$$w(t+1) = w(t) - \gamma \frac{\hat{m}_w}{\sqrt{\hat{v}_w} + \varepsilon}$$

$\beta_1 (\sim 0{,}9)$, $\beta_2 (\sim 0{,}999)$ dampening or forgetting parameters, ε is a small scalar to prevent division by 0